IACR Transactions on Symmetric Cryptology
ISSN 2519-173X, Vol. 0, No. 0, pp. 0–0. DOI:10.46586/tosc.v0.i0.0-0

Multiple Rows Mixers and Hsilu

A Family of Linear Layers and A Permutation with Fewer XORs

Xiaobin Yu[1,2] and Meicheng Liu[1,2]

[1] Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Sciences, Beijing, People's Republic of China
[2] School of Cyber Security, University of Chinese Academy of Sciences, Beijing, People's Republic of China
yuxiaobin@iie.ac.cn, liumeicheng@iie.ac.cn

Abstract. Over the past decades, extensive research has been conducted on lightweight cryptographic primitives. The linear layer plays an important role in their security. In this paper, we propose a family of linear layers consisting of XORs and rotations, which is called multiple rows mixers (MRM). It is a family designed for LS-type ciphers, but mixing elements from several rows. We investigate the impact of the linear layers on the 3-round trail weight of permutations and explore the properties of the inverse of the linear layers with a low XOR count. We employ a generic and extensible approach to determine the parameters of MRM. This approach can automatically generate linear layers that meet the requirements of a given branch number.

By applying these design principles and methods, we derive a linear layer that has a dimension of 5×64, a differential branch number of 12, a linear branch number of 5 and a computational cost of 2.6 XOR operations per bit. MRM is not limited to fixed dimension and can be extended to other dimensions. In addition, we present a concrete instantiation of a 320-bit permutation using a more efficient instance of MRM, named Hsilu. Its non-linear layer employs the χ operating on columns. Compared with the permutations of Gaston and NIST lightweight standard Ascon, the round function of Hsilu requires fewer XOR operations. Hsilu exhibits competitive security and performance with Ascon and Gaston. We demonstrate that the best-found 3-round differential and linear trails of Hsilu have much higher weights than those of Ascon. Hsilu outperforms Gaston and Ascon in terms of both software and hardware performance.

Keywords: Linear layer · Permutations · Design · Branch number · Multiple Rows Mixer · Hsilu · Lightweight

1 Introduction

In recent years, the rapid advancement of information technologies, including 5G, the Internet of Things (IoT), and cloud computing, has introduced new requirements for symmetric encryption algorithms. These technologies are typically applied in scenarios involving resource-constrained devices, such as various IoT applications, radio-frequency identification (RFID) tags, and wireless sensor networks, which have limited computational resources. The lightweight design of symmetric encryption algorithms has emerged as a significant area of research. A core challenge in this design process is how to balance the implementation performance and security of encryption algorithms in resource-constrained environments. Designers must strive to maximize security while adhering to specified implementation cost requirements. Implementation performance can be categorized into

hardware implementation performance and software implementation performance. In this paper, we focus on the total number of binary Boolean operations.

The core of designing lightweight symmetric encryption algorithms lies in the study of their components. Innovative component design is essential for addressing the new challenges that contemporary information systems present to cryptographic algorithms. There are some similarities between the designs of block ciphers and hash functions, primarily evident in the nonlinear components that provide confusion and linear components that provide diffusion. Therefore, the design of algorithm components is of paramount importance. The linear layer is a significant component, and its diffusion properties are achieved through a specific internal structure. The design of the linear layer is important, as it directly impacts the security of ciphers.

A common strategy for designing a block cipher is the alternating use of substitution layers, linear layers, and key addition. This structure is called substitution permutation network (SPN). Many widely used cryptographic algorithms adhere to this construction: AES [DR02], PRESENT [BKL+07], RECTANGLE [ZBRL15], SKINNY [BJK+16], GIFT [BPP+17]. To evaluate the security of these ciphers against differential and linear cryptanalysis, designers typically specify the upper limits on the number of active S-boxes in the differential or linear trails.

We briefly review some related work on linear layer design. Maximum-distance-separable (MDS) matrices play a significant role in the construction of block ciphers. For example, the MixColumns operation in AES uses a 4×4 MDS matrix. Its branch number is 5. The MDS matrix theoretically has the highest differential branch number and linear branch number. Investigating MDS matrices with low implementation cost is challenging. There is a lot of research on the construction of low-cost MDS matrices [KLSW17, DL18].

Ascon [DEMS21] has been selected as the NIST lightweight standard. The linear layer Ascon p_L is similar to the Σ functions in SHA-2. p_L costs 2 XOR operations per bit. However, the part of the mixing layer in the S-boxes p_S has an additional cost of 1.2 XOR operations per bit, totaling to 3.2 XOR operations per bit. In this split-up, the non-linear layer of Ascon is the χ mapping.

Spook [BBB+20] is an AEAD scheme using primitives that build upon the ideas of LS-designs. The linear layer, L-box, of the Spook operates on two words at once. Compared with Ascon p_L, Spook L-box and all the linear layers described below cannot be split into a number of parallel mappings.

Leurent and Pernot [LP24] extends the construction of the Spook linear layer by considering a linear layer that operates on 3 or 4 words at a time, named $L_{32\times3}$ and $L_{32\times4}$. Their XOR count per bit and branch number are shown in Table 1.

We compare our work with that of Leurent and Pernot [LP24]. In terms of linear layers, the XOR operations per bit of $L_{32\times3}$ and $L_{32\times4}$ are 6 which is higher than 2.6 of our work. Their branch number is also higher than ours. Our linear layer is more suitable for lightweight ciphers. In terms of approach, they use an interleaved implementation so that the differential branch number is equal to the linear branch number. We both use the search approach to search for linear layers. We use an MILP model with filters, while they use an Information Set Decoding (ISD) algorithm.

The column parity mixer (CPM) is a mixing layer presented by Stoffelen and Daemen [SD18]. CPM is a generalization of the mixing layer of Keccak-f. The mixing layer of Xoodoo also belongs to CPM. The number of XORs per bit of CPM is low as shown in Table 1. Their dimensions and branch numbers are also shown in Table 1.

The circulant twin column parity mixer (twin CPM) serves as a mixing layer for the round function of cryptographic permutations, as proposed by Hirch et al. [HDRM23] at CRYPTO 2023. It features a differential branch number of 12 and a linear branch number of 4. Hirch et al. provided a concrete instantiation of a permutation utilizing twin CPM, referred to as Gaston, and demonstrated that the best-found 3-round differential and linear

trails of Gaston possess significantly higher weights compared to those of Ascon.

Lei et al. [LRL+24] introduce the symmetric twin CPM, which improves the linear security of twin CPM. Its differential and linear branch numbers are both 12. Compared with twin CPM, its number of XORs per bit has increased from 3.2 to 4. Using symmetric twin CPM, they present two new 320-bit cryptographic permutations, namely Gaston-S and SBD. Gaston-S uses the χ as non-linear layer while SBD uses a low-latency degree-4 S-box as non-linear layer. The weights of the best-found 3-round linear trails of Gaston-S and SBD are significantly higher than those of Gaston and Ascon.

In Table 1, we compare these different types of linear layers with our work.

Table 1: A comparison of different types of linear layers

Linear layer	Dimensions	XORs per bit	Branch number		Source
			Diff.	Linear	
MDS	$3 \times 3, GF(2^d)$	$\frac{5}{3} + \frac{1}{3d}$	4	4	[DL18]
	$4 \times 4, GF(2^d)$	$2 + \frac{3}{4d}$	5	5	[DL18]
	$8 \times 8, GF(2^4)$	6.06	9	9	[KLSW17]
	$8 \times 8, GF(2^8)$	6.125	9	9	[KLSW17]
Spook L-box	$2 \times 32, GF(2)$	6	16	16	[BBB+20]
$L_{32\times3}$	$3 \times 32, GF(2)$	6	19	19	[LP24]
$L_{32\times4}$	$4 \times 32, GF(2)$	6	21	21	[LP24]
Ascon p_L	$5 \times 64, GF(2)$	2	4	4	[DEMS21]
Keccak-f linear layer	$5 \times 5 \times w^{\dagger}, GF(2)$	2	4	4	[BDPA11]
Xoodoo linear layer	$3 \times 4 \times 32, GF(2)$	2	4	4	[DHAK18]
CPM	$m \times n, GF(2)$	$2 + \frac{h-2}{m}^{\ddagger}$	4	4	[SD18]
Twin CPM	$m \times n, GF(2)$	$3 + \frac{1}{m}$	12	4	[HDRM23]
Transpose of Twin CPM	$m \times n, GF(2)$	$3 + \frac{1}{m}$	4	12	[HDRM23]
Symmetric Twin CPM	$m \times n, GF(2)$	4	12	12	[LRL+24]
MRM	$4 \times l, GF(2)$	2.75	12	5	This work
	$5 \times l, GF(2)$	2.6	10	5	
	$5 \times l, GF(2)$	2.6	12	5	
	$6 \times l, GF(2)$	2.67	13	5	

$\dagger$: w is the number of slides and $w \in \{8, 16, 32, 64\}$ based on the variant of Keccak-f. $\ddagger$: h is the Hamming weight of the parity-folding polynomial as defined in [SD18].

1.1 Motivation

We primarily focus on the performance and security characteristics of the linear layers in Ascon and Gaston. Our objective is to design linear layers that not only achieve higher implementation efficiency, but also offer improved security. In terms of implementation efficiency, we concentrate on the number of XOR operations per bit. Regarding security, we emphasize the branch number and the weight of 3-round trails. Similar to Ascon and Gaston, we primarily consider a dimension of 5×64, while also allowing for extensions to more general $r \times l$ dimensions. We are inspired to adopt a search approach for determining the parameters of the linear layers by Leurent and Pernot [LP24].

1.2 Our Contributions

1. **The multiple rows mixer.** We study the differential and linear diffusion properties of linear layers consisting of XORs and cyclic shifts. Then we design a new family of

linear layers MRM. MRM is not limited to fixed dimension and can be extended to other dimensions. One of the MRM has a dimension of 5×64, a differential branch number of 12 and a linear branch number of 5 with 2.6 XORs per bit. Compared with the linear layers of Ascon and Gaston, MRM has better branch numbers and lower cost.

2. **The linear layer featuring a low XOR count and a complex inverse.** Besides branch number, we incorporate the weight of 3-round trails as one of the metrics for evaluating the linear layers during design. By discussing how to increase the minimum 3-round trail weight, we find that we need a linear layer featuring a low XOR count and a complex inverse. The inverse is complex, which means that the inverse of the corresponding matrix has many 1 in its columns. Then we study the properties of this kind of linear layers. Based on our study, we present two patterns of this kind of linear layers.

3. **A generic and extensible algorithm for automated generation of linear layers.** We propose a generic and extensible approach for automated generation of linear layers with different numbers of XOR operations that meet the specified branch number and activity pattern requirements. The algorithm can search for linear layers efficiently.

4. **Design, security and implementation of Hsilu.** We present the design of a new 320-bit cryptographic permutation, namely Hsilu. In Hsilu, the linear layer with a differential branch number of 12 is not used; instead, a more efficient linear layer with a differential branch number of 10 and a linear branch number of 5 is used. Hsilu employs the χ mapping as the non-linear layer.

   We evaluate the differential and linear trails of Hsilu, showing that the minimum weight of 3-round differential trail is higher than that of Ascon, and the minimum weight of 3-round linear trail is higher than those of Ascon and Gaston.

   We also implement Hsilu on a hardware platform (SkyWater 130nm) and two software platforms (x86_64 and ARMv8). Compared with Ascon, Xoodoo, Gaston, the hardware area and gates of Hsilu are smaller. The time delay of Hsilu is less than those of Ascon and Gaston. In terms of software performance, the speed (measured in cycles per round per byte) of Hsilu outperforms those of Ascon and Gaston.

1.3 Outline

In Section 2, we recall the differential propagation, linear propagation, the wide-trail strategy and the definition of the branch number. In Section 3, we describe our linear layer, multiple rows mixers. In Section 4, we discuss the diffusion metrics about 3-round trail and how to reduce minimum weight of 3-round trail. In Section 5, we study the property of the inverse of lightweight linear layer. In Section 6, we discuss the differential and linear diffusion in MRM. In Section 7, we describe the permutation Hsilu and finalize the parameters of Hsilu. In Section 8, we give the trails bound and implementation results. In Section 9, we provide conclusions and open problems.

2 Preliminaries

2.1 Differential Cryptanalysis

Differential cryptanalysis was introduced by Biham and Shamir in [BS91]. Let x_1 and x_2 be two inputs to a transformation f over $\mathbb{F}_2^n$, and $y_1 = f(x_1)$ and $y_2 = f(x_2)$ be the outputs. $a = x_1 \oplus x_2$ is an input difference of f and $b = y_1 \oplus y_2$ is an output difference.

Their combination (a, b) is called a differential over f. The differential probability (DP) of a differential (a, b) is defined as

$$\mathrm{DP}(a,b) = \frac{\#\{x \in \mathbb{F}_2^n \mid f(x) \oplus f(x \oplus a) = b\}}{2^n}. \tag{1}$$

The *restriction weight* w_r of a differential is defined by $\mathrm{DP} = 2^{-w_r}$.

Let α be an iterative mapping: $\alpha = p_r \circ \cdots \circ p_2 \circ p_1$. We call a differential over the round function a round differential. Round differentials can be linked to form a differential trail. An r-round differential trail Q is defined by a sequence of difference patterns after each round $(a^0, a^1, \ldots, a^r)$. The DP of a trail is challenging to calculate and often approximated by its expected DP (EDP). The EDP of a differential trail is defined as the differential probability of the trail averaged on all keys. The weight of a differential trail is denoted by $w_r(Q)$ and we have $w_r(Q) = -\log_2 \mathrm{EDP}(Q) = -\log_2(\prod_{0<i\leq r} \mathrm{DP}(a^{i-1}, a^i)) = \sum_{0<i\leq r} w_r(a^{i-1}, a^i)$.

2.2 Linear Cryptanalysis

Linear cryptanalysis was introduced by Matsui in [Mat94]. We consider input mask a and output mask b instead of differences. (a, b) is called a linear approximation over f. The correlation C of a linear approximation (a, b) is defined as:

$$\mathrm{C}(a,b) = \frac{\#\{x \in \mathbb{F}_2^n \mid \langle a, x\rangle = \langle b, f(x)\rangle\}}{2^{n-1}} - 1. \tag{2}$$

The *correlation weight* w_c of a linear approximation is defined by $\mathrm{C}^2 = 2^{-w_c}$.

Let α be an iterative mapping: $\alpha = p_r \circ \cdots \circ p_2 \circ p_1$. We call a linear approximation over the round function a round linear approximation. Round linear approximations can be linked to form a linear trail. Similar to a differential trail, an r-round linear trail Q is defined by a sequence of linear masks before and after each round $(a^0, a^1, \ldots, a^r)$. The correlation C of a trail is defined as $\mathrm{C}(Q) = \prod_{0<i\leq r} C(a^{i-1}, a^i)$. The weight of a linear trail $w_c(Q)$ is defined as $w_c(Q) = -\log_2 \mathrm{C}^2(Q) = \sum_{0<i\leq r} w_c(a^{i-1}, a^i)$.

2.3 Branch Number

The wide-trail strategy [DR01] is an important strategy for designing ciphers. This strategy requires that, when designing a cipher, a large number of active S-boxes should be ensured, with these S-boxes having good cryptographic properties. Non-zero S-boxes or non-zero bits are referred to as active S-boxes or active bits. $w(x)$ denotes the number of active S-boxes of a state x and $w_b(x)$ denotes the number of active bits of a state x.

The concept of branch number is an important security property, measuring the diffusion capability of a linear layer.

Definition 1 (Differential branch number). For a linear layer L, its differential branch number is denoted by $\mathcal{B}_d(L)$ and defined as

$$\mathcal{B}_d(L) = \min_{x\neq 0}(w(L(x)) + w(x)). \tag{3}$$

Definition 2 (Bit differential branch number). For a linear layer L, its bit differential branch number is denoted by $\mathcal{B}_{d,bit}(L)$ and defined as

$$\mathcal{B}_{d,bit}(L) = \min_{x\neq 0}(w_b(L(x)) + w_b(x)). \tag{4}$$

The differential branch number can be simply called the branch number.

Definition 3 (Linear branch number). For a linear layer L, its linear branch number is denoted by $\mathcal{B}_l(L)$ and defined as

$$\mathcal{B}_l(L) = \min_{x \neq 0}(w(L^{\mathsf{T}}(x)) + w(x)). \tag{5}$$

Definition 4 (Bit linear branch number). For a linear layer L, its bit linear branch number is denoted by $\mathcal{B}_{l,bit}(L)$ and defined as

$$\mathcal{B}_{l,bit}(L) = \min_{x \neq 0}(w_b(L^{\mathsf{T}}(x)) + w_b(x)). \tag{6}$$

The L^{T} is the linear transformation whose matrix representation over $\mathbb{F}_2$ is the transpose of the matrix representation of L.

3 Multiple Rows Mixer

We use a sequence of XORs and rotations to construct a linear layer. This linear layer is called *multiple rows mixer* or MRM for short. MRM has a dimension of $r \times l$. r is the number of words and l is the width of the word. This design facilitates the bitsliced implementation of ciphers. The MRM consists of two steps θ, ρ:

$$p_M = \rho \circ \theta. \tag{7}$$

Algorithm 1 Multiple rows mixer

Input: $(x_0, x_1, \ldots, x_{r-1})$
for j from r to $r + t - 1$ **do**
  $x_j \leftarrow x_{n_1[j]} \oplus (x_{n_2[j]} \lll m_j)$ ▷ θ
end for
for j from t to $r + t - 1$ **do**
  $x_j \leftarrow (x_j \lll s_j)$ ▷ ρ
end for
return $(x_t, x_{t+1}, \ldots, x_{r+t-1})$

3.1 Shifted Variants of XOR

The operation $x \oplus (y \lll z)$ is called shifted variant of XOR or shifted XOR. t is the number of XOR operations. This step involves t shifted XORs and is referred to as θ:

$$\theta : x_i \leftarrow x_{n_1[i]} \oplus (x_{n_2[i]} \lll m_i),\ r \leq i \leq r + t - 1. \tag{8}$$

We are given $x_i\,(0 \leq i < r)$ as the input. Each $x_i\,(r \leq i \leq r + t - 1)$ is computed iteratively using the formula 8. The $x_{n_1[i]}$ and $x_{n_2[i]}$ represent either previously calculated values or known inputs. $0 \leq n_1[i] < i$ and $0 \leq n_2[i] < i$. m_i denotes the offset of cyclic shift. $0 \leq m_i < l$.

The mixing layer θ provides diffusion in r words. It has a computational cost of $\frac{t}{r}$ XORs per bit. This kind of operation is efficient. In some software implementations, we can use a single instruction to perform a shifted XOR operation, such as the implementation of MRM on the ARMv8 instruction set architecture. The ARM's barrel shifter allows instructions to apply a range of constant shifts to the second operand register.

3.2 Cyclic Row Shift

The cyclic row shift step is referred to as ρ:

$$\rho : x_i \leftarrow (x_i \lll s_i),\ t \leq i \leq r + t - 1. \tag{9}$$

The $(x_i \lll s_i)$ denotes x_i rotated left by s_i bits. The row shift step can diffuse the active bits into different S-boxes. If we do not use ρ, the linear layer may encounter a situation in which multiple active bits are in the same S-box. This will cause the bit branch number to be high while the branch number to be low.

In fact, it is only the difference between the offsets s_i that matters and therefore we can set $s_t = 0$. We need to specify $r - 1$ offsets.

3.3 Tabular Representation

The linear layer MRM can be represented by four sequences n_1, n_2, m, s. Table 2 represents an instance of MRM, using 13 XOR operations with the word width of 64 bits. The input is $(x_0, x_1, x_2, x_3, x_4)$, and the output is $(x_{13}, x_{14}, x_{15}, x_{16}, x_{17})$. Its differential branch number is 12.

Table 2: Tabular representation of an instance of MRM

i	5	6	7	8	9	10	11	12	13	14	15	16	17
$n_1[i]$	1	2	2	0	5	5	6	11	12	13	14	15	16
$n_2[i]$	4	3	1	3	0	4	9	7	8	10	11	12	9
m_i	15	34	9	16	27	14	61	13	42	43	43	52	41
s_i									0	57	13	15	22

3.4 Matrix Corresponding to Linear Layer

Let the linear layer be denoted by L and the matrix corresponding to the linear layer be denoted by M. M has a dimension of $rn \times rn$ over $\mathbb{F}_2$. Due to the cyclic property of the shifted XOR and cyclic row shift operations, the matrix M can be partitioned into r^2 circulant matrices. Each line of circulant matrices is a rotation of the previous line.

$$(x_t, x_{t+1}, \cdots, x_{t+r-1}) = L(x_0, x_1, \cdots, x_{r-1}) \tag{10}$$

$$(x_t, x_{t+1}, \cdots, x_{t+r-1})^T = M \times (x_0, x_1, \cdots, x_{r-1})^T \tag{11}$$

$$M = \begin{bmatrix} A_{0,0} & A_{0,1} & \cdots & A_{0,r-1} \\ A_{1,0} & A_{1,1} & \cdots & A_{1,r-1} \\ \vdots & \vdots & \ddots & \vdots \\ A_{r-1,0} & A_{r-1,1} & \cdots & A_{r-1,r-1} \end{bmatrix} \tag{12}$$

where $A_{i,j}$ is an $n \times n$ circulant matrix.

4 Diffusion Metrics of 3-Round Trails

In the previous analyses of the security of linear layers, attention has generally been focused on their differential and linear branch numbers. For an SPN cipher composed of linear layers and S-boxes, the minimum weight of the 2-round trails has a strong relationship with the differential branch number and the linear branch number of the linear layer. The weight also depends on the specific non-linear layer. However, the branch number alone

cannot fully determine the weight of longer trails. In this paper, we incorporate the weight of 3-round trails as one of the metrics for evaluating the linear layers during design. The analysis of trails with more than three rounds is more challenging, hence we focus on the 3-round trails.

We observe that for a linear layer that is not elaborately designed, when the weight of the 3-round trail of the corresponding SPN cipher is at its minimum, it tends to have a low weight (usually 1 or 2) in the second round. This observation is reasonable. We use (m, n) to represent a trail with m and n active S-boxes in the first round and in the second round. Two low-weight 2-round trails, (m, a) and (a, n), can easily concatenate to form a 3-round trail (m, a, n) with high probability with a small a.

To prevent the lowest weight of the 3-round trail from being (m, a, n), we need to maximize m or n for the trails (m, a) and (a, n) with a small a. Since the number of XOR operations is limited, increasing n is not easy, but it is possible to make m significantly high. This suggests that the inverse of the linear layer is complex. In other words, the inverse of this linear layer corresponds to a dense matrix.

We use the activity pattern to facilitate research like [MR22]. Specifically, for a linear layer L and a given number of active input S-boxes d, $N_{min}[d]$ and $N_{max}[d]$ denote the minimum and maximum number of active output S-boxes after L. Similarly, the minimum and maximum number of active output S-boxes after L^{-1} are denoted by $N^{-1}_{min}[d]$ and $N^{-1}_{max}[d]$. $N_{min}[d]$, $N_{max}[d]$, $N^{-1}_{min}[d]$ and $N^{-1}_{max}[d]$ are called the activity pattern of a linear layer. The activity pattern provides a clearer and more intuitive framework than the branch number for studying the properties of the linear layer.

The general idea of our construction is adopting a composition of two linear layers, which will be discussed in the following sections. In Section 5, we first investigate the linear layer, denoted as p_1, for values of $N^{-1}_{min}[1]$ greater than 4. Section 6 focuses on the analysis of differential diffusion and linear mask propagation. By composing an additional linear layer p_2 with p_1, we can increase both $N^{-1}_{min}[1]$ and $N^{-1}_{min}[2]$. The construction is then finalized in Section 7.

5 It is Simple, but the Inverse is Complex

In this section, we focus on the minimum number of active bits for MRM in terms of differentials. Analyzing the minimum number of active input bits is more convenient than studying the number of active input S-boxes. Therefore, we denote $\mathcal{N}_L$ as the minimum number of active input bits for the linear layer L when the number of active output bits is 1. We have $\mathcal{N}_L \geq N^{-1}_{min}[1]$. Now, our focus shifts to active bits rather than active S-boxes. We aim to maximize $\mathcal{N}_L$. In the subsequent analysis, we assume that $(x_0, x_1, \ldots, x_{r-1})$ represents the input and the last $(x_t, x_{t+1}, \ldots, x_{r+t-1})$ represents the output.

Based on our research and analysis, we summarize the following proposition, lemma, and corollaries.

Proposition 1. *For any invertible linear layer* $L : (x_0, x_1, \ldots, x_{r-1}) \mapsto (x_r, x_{r+1}, \ldots, x_{2r-1})$, *composed of* r *shifted XORs:*

$$x_i \leftarrow x_{n_1[i]} \oplus (x_{n_2[i]} \lll m_i),\ r \leq i \leq 2r-1, \tag{13}$$

we conclude that $\mathcal{N}_L \leq 2$.

Proof. We define a multiset $S = \{n_1[i], n_2[i]\}_{r \leq i \leq 2r-1}$. We classify based on the count of values greater than $r-1$ in S, denoting this count as $N = \#\{s \in S \mid s > r-1\}$.

Case 1: $N = 0$.

When the input bits are all 1, i.e., $x_i = 11\ldots11$ for $0 \leq i \leq r-1$, the output bits are all 0. Thus, this linear layer is not invertible.

Case 2: $N \geq 1$.

Assume the minimum element greater than $r-1$ of S is a and the amount of a in S is j. Without loss of generality, we can take $n_1[a] = p < r$, $n_2[a] = q < r$, $n_1[v_0] = a, \ldots, n_1[v_{j-1}] = a$. Then $n_2[v_i] \leq r-1$; otherwise, the outputs are linearly dependent, implying that L is not invertible.

Let $x_a = 0$; then $x_p = x_q \lll m_a$. We change all x_q to $x_p \ggg m_a$ when calculating the XORs. Then we delete x_q in input and delete x_a in output. We change $x_{v_i} = x_a \oplus x_{n_2[v_i]} \lll m_{v_i}$ to $x_{v_i} = 0 \oplus x_{n_2[v_i]} \lll m_{v_i} = x_{n_2[v_i]} \lll m_{v_i}$. We can get a new linear layer $L_1 : (x_0, \ldots, x_{q-1}, x_{q+1}, \ldots, x_{r-1}) \mapsto (x_r, \ldots, x_{a-1}, x_{a+1}, \ldots, x_{2r-1})$.

We can conclude that $\mathcal{N}_L \leq \mathcal{N}_{L,x_a=0} \leq \mathcal{N}_{L_1} + \#\{active\ bits\ in\ x_p\} \leq 2 \times \mathcal{N}_{L_1}$.

If L_1 is is not invertible then there exists a non-zero input A such that $L_1(A) = 0$. By concatenating this input with $x_q = x_p \ggg m_a$, we obtain a non-zero input $A' = (A, x_q = x_p \ggg m_a)$ for which $L(A') = 0$. Since if L_1 is is not invertible, L is also not invertible.

For convenience, L_1 can be regarded as linear layer $L_2 : (x_0, \ldots, x_{k-1}) \mapsto (x_k, \ldots, x_{2k-1})$. $U = \{u_i\}_{0 \leq i < j}$. L_2 is composed of $k-j$ shifted XORs and j cyclic shifts:

$$x_i \leftarrow x_{n_1[i]} \oplus (x_{n_2[i]} \lll m_i),\ k \leq i \leq 2k-1,\ i \notin U, \tag{14}$$

$$x_i \leftarrow x_{n_2[i]} \lll m_i,\ i \in U. \tag{15}$$

Now we prove that $\mathcal{N}_{L_2} = 1$ for all L_2. For L_2, we define $S_2 = \{n_1[i], n_2[i]\}_{k \leq i \leq 2k-1,\ i \notin U}$.

Case 2.1: $\exists i < j,\ u_i \notin S_2,\ n_2[u_i] \notin S_2$.

Let $x_{n_2[u_i]} = 00\ldots01$; then $\mathcal{N}_{L_2} = 1$.

Case 2.2: $\forall i < j,\ u_i$ *or* $n_2[u_i] \in S_2$.

Aside from u_i or $n_2[u_i]$, there are at most $2(k-j)-j$ remaining positions in S_2. Define $NU = \{n_2[u_i]\}_{0 \leq i < j}$. Assuming the opposite that $\mathcal{N}_{L_2} > 1$, then for arbitrary $c \in [0, k-1]$ and $c \notin NU$, input x_c must effect at least two outputs. This leads to three cases.

Case 2.2.1: $\#\{s \in S_2 \mid s = c\} > 1$.

In this case, c ($c \notin NU$) appears at least twice in the remaining $2k-3j$ positions of S_2.

Case 2.2.2: $\#\{s \in S_2 \mid s = c\} = 1$. $n_1[e] = c$, $n_2[e] = d > k-1$ or $n_2[e] \in NU$. $\#\{s \in S_2 \mid s = e\} \geq 1$.

In this case, c and e (c, $e \notin NU$) use at least two positions in S_1, where x_e is not input. We can know one input x_c needs at least two positions.

Case 2.2.3: $\#\{s \in S_2 \mid s = c\} = 1$. $n_1[e] = c$, $n_2[e] = d \leq k-1$ and $d \notin NU$. $\#\{s \in S_2 \mid s = e\} \geq 1$.

In this case, x_c and x_d are inputs. c or d must appear again; otherwise, if we let $x_c = x_d = 11\ldots1$ and other inputs are all 0, then the outputs are all 0, implying that L_2 is not invertible. c, d and e use at least four positions in S_2. We know that two inputs x_c and x_d need at least four positions.

Based on these three cases, we conclude that at least two positions in S_2 are required for one input; otherwise $\mathcal{N}_{L_2} = 1$. However, $2k-3j$ positions are not enough for $k-j$ inputs (excluding $x_{n_2[u_i]}$). Therefore, $\mathcal{N}_L \leq 2 \times \mathcal{N}_{L_1} = 2 \times \mathcal{N}_{L_2} = 2$. □

Lemma 1. *For an invertible linear layer* $L : (x_0, x_1, \ldots, x_{r-1}) \mapsto (x_{r+1}, x_{r+2}, \ldots, x_{2r})$, *composed of* $r+1$ *shifted XORs:*

$$x_i \leftarrow x_{n_1[i]} \oplus (x_{n_2[i]} \lll m_i),\ r \leq i \leq 2r, \tag{16}$$

define $S = \{n_1[i], n_2[i]\}_{r+1 \leq i \leq 2r}$. *If* $\#\{s \in S \mid s > r\} \geq 1$, *then* $\mathcal{N}_L \leq 4$.

Proof. Assume the minimum element greater than r of S is b and the amount of b in S is j. Without loss of generality, we can take $n_1[b] = p \leq r$, $n_2[b] = q \leq r$, $n_1[v_0] = b, \ldots, n_1[v_{j-1}] = b$. We have $n_2[v_i] \leq r$, otherwise, $(x_{r+1}, x_{r+2}, \ldots, x_{2r})$ is linearly dependent which implies that L is non-invertible.

Assume $p = q = r$. $n_1[r]$ and $n_2[r]$ must each appear at least once in S, otherwise $\mathcal{N}_L \leq 2$. Aside from $p, q, b, n_1[r], n_2[r]$, there are at most $2r-5$ positions. Similar to the proof in Case 2.2 of Proposition 1, we can know that $2r-5$ positions is not enough for other $r-2$ inputs. This means $\mathcal{N}_L \leq 1$. So we consider the case where $q < r$.

Let $x_b = 0$; then $x_p = x_q \lll m_b$. We change all x_q to x_p when calculating the XORs. Then we delete x_q in input and delete x_b in output. We change $x_{v_i} = x_b \oplus x_{n_2[v_i]} \lll m_{v_i}$ to $x_{v_i} = 0 \oplus x_{n_2[v_i]} \lll m_{v_i} = x_{n_2[v_i]} \lll m_{v_i}$. We can get a new linear layer $L_1 : (x_0, \ldots, x_{q-1}, x_{q+1}, \ldots, x_{r-1}) \mapsto (x_{r+1}, \ldots, x_{b-1}, x_{b+1}, \ldots, x_{2r})$. We can conclude that $\mathcal{N}_L \leq \mathcal{N}_{L,x_b=0} \leq \mathcal{N}_{L_1} + \#\{active\ bits\ in\ x_p\} \leq 2 \times \mathcal{N}_{L_1}$.

For convenience, L_1 can be regarded as linear layer $L_2 : (x_0, \ldots, x_{k-1}) \mapsto (x_{k+1}, \ldots, x_{2k})$. $U = \{u_i\}_{0 \leq i < j}$. L_2 is composed of $k+1-j$ shifted XORs and j cyclic shifts:

$$x_i \leftarrow x_{n_1[i]} \oplus (x_{n_2[i]} \lll m_i),\ k \leq i \leq 2k,\ i \notin U, \tag{17}$$

$$x_i \leftarrow x_{n_2[i]} \lll m_i,\ i \in U. \tag{18}$$

Now we prove that $\mathcal{N}_{L_2} \leq 2$ for all L_2. For L_2, we define $S_2 = \{n_1[i], n_2[i]\}_{k \leq i \leq 2k,\ i \notin U}$.

Case 1: $\exists u_i \in U,\ n_2[u_i] = k$.

$x_{u_i} = x_k \lll m_{u_i}$. We can calculate x_k when calculating x_{u_i}, which is equivalent to eliminating the calculation of x_k. Then this linear layer is equivalent to the situation in Proposition 1, where only $k-j$ XORs are used. By Proposition 1, We can conclude that $\mathcal{N}_{L_2} \leq 2$. Thus, $\mathcal{N}_L \leq 2 \times \mathcal{N}_{L_2} \leq 4$.

Case 2: $\forall u_i \in U,\ n_2[u_i] = c < k$.

Case 2.1: $\forall s \in S_2,\ s \leq k$.

$u_0, \ldots, u_{j-1}$ must each appear at least once in S_2, otherwise $\mathcal{N}_{L_2} = 1$. If $n_1[k] \neq n_2[k]$, $k, n_1[k], n_2[k]$ must appear at least three times in total in S_2, otherwise $\mathcal{N}_{L_2} = 1$. If $n_1[k] = n_2[k]$, $k, n_1[k], n2[k]$ must appear at least twice in total in S_2, otherwise $\mathcal{N}_{L_2} = 1$. Other inputs $x \notin \{u_0, \ldots, u_{j-1}, n_1[k], n_2[k]\}$ must each appear at least twice in S_2, otherwise $\mathcal{N}_{L_2} = 1$. There are $2(k-j) - j - k_1$ remaining positions for other $k - j - k_2$ inputs. So $j \leq 2k_2 - k_1$.

If $n_1[k] \neq n_2[k]$ and $n_1[k], n_2[k] \notin U$, then $k_1 = 3, k_2 = 2$.

If $n_1[k] \neq n_2[k]$ and one of $n_1[k], n_2[k] \in U$, then $k_1 = 2, k_2 = 1$.

If $n_1[k] \neq n_2[k]$ and both $n_1[k]$ and $n_2[k] \in U$, then $k_1 = 1, k_2 = 0$.

If $n_1[k] = n_2[k]$ and $n_1[k] \notin U$, then $k_1 = 2, k_2 = 1$.

If $n_1[k] = n_2[k]$ and $n_1[k] \in U$, then $k_1 = 1, k_2 = 0$.

Based on these cases, we can get $n_1[k] \neq n_2[k]$, $n_1[k], n_2[k] \notin U$ and $j \leq 1$. So $j = 1$. $k, n_1[k], n_2[k]$ must each appear at least once in S_2, otherwise $\mathcal{N}_{L_2} \leq 2$. When $x_i \leftarrow x_{n_1[i]} \oplus (x_{n_2[i]} \lll m_i)$, we define $n_1[i]$ and $n_2[i]$ as each other's partners. Assume c's partner is d_0. If $d_i \notin \{k, n_1[k], n_2[k]\}$, d_i has another partner d_{i+1}. Assume $d_i \notin \{k, n_1[k], n_2[k]\}$ for $i < h$ and $d_h = k$ or $n_1[k]$ or $n_2[k]$. Let $c = d_0 = \cdots = d_h = 0$ and other inputs are $11\ldots1$. Then the output of L_2 are all 0. L_2 is not invertible.

Case 2.2: $\exists s \in S_2,\ s > k$.

Assume the minimum element greater than k of S_2 is s' and the amount of s' in S_2 is j'. Without loss of generality, we can take $n_1[s'] = p' \leq k$, $n_2[s'] = q' < k$, $n_1[u'_0] = b', \ldots, n_1[u'_{j'-1}] = b'$. $U' = \{u'_i\}_{0 \leq i < j'}$. Let $x_{s'} = 0$; then $x_{p'} = x_{q'} \lll m_{s'}$. We change all $x_{q'}$ to $x_{p'}$ when calculating the XORs. Then we delete $x_{q'}$ in input and delete $x_{s'}$ in output. We change $x_{u'_i} = x_{s'} \oplus x_{n_2[u'_i]} \lll m_{u'_i}$ to $x_{u'_i} = 0 \oplus x_{n_2[u'_i]} \lll m_{u'_i} = x_{n_2[u'_i]} \lll m_{u'_i}$. We can get a linear layer $L_3 : (x_0, \ldots, x_{q'-1}, x_{q'+1}, \ldots, x_{k-1}) \mapsto (x_{k+1}, \ldots, x_{s'-1}, x_{s'+1}, \ldots, x_{2k})$. We can conclude that $\mathcal{N}_{L_2} \leq \mathcal{N}_{L_2, x_{s'}=0} \leq 2 \times \mathcal{N}_{L_3}$. Similar to the proof in Case 2.1, we can get $j + j' \leq 1$, otherwise the positions is not enough. $j + j' \leq 1$ is impossible. So $\mathcal{N}_{L_3} = 1$. Then $\mathcal{N}_{L_2} \leq 2$.

In conclusion, we have $\mathcal{N}_L \leq 4$. □

Corollary 1. *For an invertible linear layer* $L : (x_0, x_1, \ldots, x_{r-1}) \mapsto (x_{r+1}, x_{r+2}, \ldots, x_{2r})$, *composed of* $r+1$ *shifted XORs:*

$$x_i \leftarrow x_{n_1[i]} \oplus (x_{n_2[i]} \lll m_i),\ r \le i \le 2r, \tag{19}$$

define $S = \{n_1[i], n_2[i]\}_{r+1 \le i \le 2r}$. *If* $\#\{s \in S \mid s > r-1\} = 1$ *and* $\mathcal{N}_L > 4$, *then that element greater than* $r-1$ *must be* r.

Proof. By Lemma 1, we can conclude that if $\#\{s \in S \mid s > r-1\} = 1$ and $\mathcal{N}_L > 4$, then that element greater than $r-1$ must be r. □

Corollary 2. *For an invertible linear layer* $L : (x_0, x_1, \ldots, x_{r-1}) \mapsto (x_{r+1}, x_{r+2}, \ldots, x_{2r})$, *composed of* $r+1$ *shifted XORs:*

$$x_i \leftarrow x_{n_1[i]} \oplus (x_{n_2[i]} \lll m_i),\ r \le i \le 2r, \tag{20}$$

define $S = \{n_1[i], n_2[i]\}_{r+1 \le i \le 2r}$. *If* $\#\{s \in S \mid s > r-1\} > 2$, *then* $\mathcal{N}_L \le 4$.

Proof. Assume that $\#\{s \in S \mid s > r-1\} > 2$. By Lemma 1, we just need to prove the case when they are all r.

When there are at least three r in S, assume $n_1[r] = a, n_2[r] = b$. If $a \neq b$, a and b appear at least twice in total; otherwise, it is not invertible or $\mathcal{N}_L = 1$. $2r-3-2 = 2r-5$ positions are not enough for $r-2$ inputs. If $f = g$, f appears at least once; otherwise, it is not invertible. $2r-3-1 = 2r-4$ positions are not enough for $r-1$ inputs. The a and b must appear at least once in n_1 and n_2, otherwise $\mathcal{N}_L = 1$. There are at most $2r-3-2 = 2r-5$ positions left. There are $r-2$ inputs other than a and b. So $\mathcal{N}_L = 1$. □

Corollary 3. *For an invertible linear layer* $L : (x_0, x_1, \ldots, x_{r-1}) \mapsto (x_{r+1}, x_{r+2}, \ldots, x_{2r})$, *composed of* $r+1$ *shifted XORs:*

$$x_i \leftarrow x_{n_1[i]} \oplus (x_{n_2[i]} \lll m_i),\ r \le i \le 2r, \tag{21}$$

define $S = \{n_1[i], n_2[i]\}_{r+1 \le i \le 2r}$. *If* $\#\{s \in S \mid s > r-1\} = 2$ *and* $\mathcal{N}_L > 4$, *then the two elements greater than* $r-1$ *must be* r *and their indexes* i *are different.*

Proof. By Lemma 1, we can conclude that if $\#\{s \in S \mid s > r-1\} = 2$ and $\mathcal{N}_L > 4$, then the two elements greater than $r-1$ must both be r.

Assume that the two r have the same index i. Then let the input bits be all 1. The output bits will be all 0. L is not invertible. □

Ultimately, based on the above proposition, lemma, and corollaries, we conclude that at least $r+1$ shifted XORs are required to achieve a linear layer for $\mathcal{N}_L > 4$, and the corresponding linear layers are equivalent to the two patterns in Table 3 when we use $r+1$ shifted XORs.

Table 3: Two patterns of linear layer. $\{\ldots\}$ denotes a set, and the parameters at these positions are elements of this set.

i	r	$r+1$	$\ldots$	$2r-1$	$2r$
$n_1[i]$	$\{0,\ldots,r-1\}$			r	r
$n_2[i]$	$\{0,\ldots,r-1\}$				

i	r	$r+1$	$\ldots$	$2r-1$	$2r$
$n_1[i]$	$\{0,\ldots,r-1\}$				r
$n_2[i]$	$\{0,\ldots,r-1\}$				

6 Construction of MRM

In Section 5, we investigate linear layers with a value of $\mathcal{N}_L$ greater than 4. Using $r+1$ shifted XOR operations, we derived the linear layers in Table 3 which we denote as p_1. A high value of $N_{min}^{-1}[1]$ implies that $\mathcal{N}_L$ must also be high. This enables us to eliminate certain linear layers with low values of $N_{min}^{-1}[1]$, thereby reducing the search space. For example, consider $r = 5$. We implemented a specific algorithm to search for p_1 with a large $N_{min}^{-1}[1]$, as presented in Algorithm 2. The function $weight(c)$ denotes the number of 1 in c. We then analyzed its activity pattern, as shown in Table 4. While the value of $N_{min}^{-1}[1]$ is high, the value of $N_{min}^{-1}[2]$ is just 1. Furthermore, both the differential and the linear branch numbers for this linear layer are notably low. Our objective is to increase $N_{min}^{-1}[2]$ and the branch number. Our strategy is to concatenate another linear layer p_2 to enhance its diffusion power. We will analyze the reasons why $N_{min}^{-1}[2]$ can increase. $N_{min}^{-1}[2]$ is related to the number of 1 in the XOR of any two columns of the corresponding matrix. Consider a dense matrix A (with a high number of 1 in each column) multiplied by another matrix B. We require the product to maintain a large number of 1 in the XOR of any two columns. This requirement is not overly stringent. We can use a computer search to find a suitable matrix. Thus, we can increase $N_{min}^{-1}[2]$ by concatenating another linear layer. We investigate the linear layer from both differential and linear perspectives.

Algorithm 2 Search for p_1

Input: n (Desired $N_{min}^{-1}[1]$)
for i from 0 to S **do**
  Randomly generate n_1, n_2, m.
  Calculate the matrix M corresponding to the linear layer.
  Calculate the inverse of M by Gauss-Jordan Elimination.
  $C = set\ of\ columns\ of\ M^{-1}$
  $flag = 1$
  for c in C **do**
    if $weight(c) < n$ **then**
      $flag = 0$
      break
    end if
  end for
  if $flag\ is\ 1$ **then**
    return n_1, n_2, m
  end if
end for

6.1 Differential Diffusion Properties

We aim for a small number of bits to rapidly diffuse to a large number of bits. A straightforward and intuitive approach is to employ linear layers as described in Table 5. $n_2[i] = i - 1$ or $n_1[i] = i - 1$. Each step utilizes x_{i-1} from the previous step, ensuring effective utilization of every XOR operation.

These two patterns are equivalent in terms of security, but they are not equivalent in terms of performance in software implementation on some architectures. In the first pattern, one instruction computes x_i, and the next instruction performs a rotation on x_i. The rotation operation can only be executed after the computation of x_i by the previous instruction. In contrast, for the second pattern, one instruction computes x_i , and the subsequent instruction performs a rotation on a different x_j. This reduces data conflicts

Table 4: Activity pattern of p_1

d	$N_{min}[d]$	$N_{max}[d]$	$N^{-1}_{min}[d]$	$N^{-1}_{max}[d]$
1	2	12	50	64
2	2	24	1	64
3	2	34	1	64
4	2	43	1	64
5	2	50	1	64
6	2	56	1	64
7	3	60	1	64
8	3	63	1	64
9	3	64	1	64
10	3	64	1	64

Table 5: Two patterns of p_2

i	r	$r+1$	$\dots$	$r+t-1$
$n_1[i]$			$\dots$	
$n_2[i]$		r	$\dots$	$r+t-2$

i	r	$r+1$	$\dots$	$r+t-1$
$n_1[i]$		r	$\dots$	$r+t-2$
$n_2[i]$			$\dots$	

between instructions and enhances instruction-level parallelism. Thus, we choose the second pattern.

Based on the analysis in Section 4, we should avoid allowing weight 1 to serve as a connector for 3-round trails. Since the $N^{-1}_{min}[1]$ of p_1 is high while the $N_{min}[1]$ of p_1 is low, we should concatenate a linear layer after p_1, rather than before it. In the following discussion, we investigate the composition of p_1 and p_2, $p_2 \circ p_1$, as shown in Table 6.

Table 6: The composition of p_1 and p_2. $\{\dots\}$ denotes a set, and the parameters at these positions are elements of this set.

i	r	$r+1$	$\cdots$	$2r-1$	$2r$	$2r+1$	$2r+2$	$\cdots$	$r+t-1$
$n_1[i]$	$\{0,\cdots,r-1\}$				r		$2r+1$	$\cdots$	$r+t-2$
$n_2[i]$	$\{0,\cdots,r-1\}$								

6.2 Linear Mask Propagation Properties

Since $n_1[i] = i-1$ for $i \geq 2r+2$, we have $x_{i-1} \oplus x_i = x_{n_2[i]} \lll m[i]$. Then each $n_2[i]$ for $t+1 \leq i \leq r+t-1$, is the XOR result (after cyclic shift) of two adjacent outputs.

Assume that one of the last $r-1$ values of n_2 is greater than t. This implies that the XOR result of the two adjacent outputs is another output. The r outputs will be linearly dependent. Therefore, in order to ensure invertibility, the last $r-1$ values of n_2 must be less than t.

Furthermore, we aim for the linear branch number of the linear layer $p_2 \circ p_1$ to exceed 4. For the first pattern of p_1 in Table 3, the last $r-1$ values of n_2 need to be greater than $2r-2$; otherwise, the relation $x_a \oplus x_b = x_c \oplus x_d$ occurs (x_a, x_b are cyclic shifted inputs and x_c, x_d are cyclic shifted outputs), resulting in linear branch number of 4. Thus, the last $r-1$ values of n_2 must be greater than $2r-2$ and less than t. This implies that the total number of XORs in the entire linear layer is at least $3r-2$. Similarly, for the second pattern of p_1, the total number of XORs in the linear layer is at least $3r-1$. Therefore, we choose the first pattern of p_1 in Table 3. The minimum value of the XOR count is $3r-2$, i.e., $t \geq 3r-2$. The linear layer is illustrated as shown in Table 7.

Table 7: Multiple rows mixer. $\{\dots\}$ denotes a set, and the parameters at these positions are elements of this set.

i	r	$r+1$	$\cdots$	$2r-1$	$2r$	$2r+1$	$2r+2$	$\cdots$	$t+1$	$\cdots$	$r+t-1$
$n_1[i]$	$\{0,\cdots,r-1\}$			r	r		$2r+1$	$\cdots$	t	$\cdots$	$r+t-2$
$n_2[i]$	$\{0,\cdots,r-1\}$								$\{2r-1,\cdots,t-1\}$		

6.3 The More Efficient Variant of p_2

The number of XOR operations per bit does not fully capture the actual implementation performance. In p_2, the x_i depends on x_{i-1} which causes more severe data conflicts in software implementations and a greater circuit depth in hardware implementations. We can construct a new optimized version of p_2 in which some x_i depend on x_{i-2} instead of x_{i-1}. This approach will enhance the actual implementation performance; however, it comes at the cost of a reduction in the differential branch number. This represents a trade-off between performance and security.

7 Application: the Design of Hsilu

In this section, we construct a permutation to demonstrate the effectiveness of Multiple Rows Mixers. We utilize MRM as the linear layer in the round function. Our permutation operates on a state with the same dimension as Ascon-p and Gaston. We refer to it as Hsilu. We also investigate MRM variants.

7.1 Permutation: Hsilu

Hsilu is a permutation operating on 320-bit states. For the description and application of the round transformations, the 320-bit state S is split into five 64-bit words. The permutation iteratively applies an SPN-based round transformation p that in turn consists of four steps $\rho, \theta, \chi, \iota$:

$$p = \rho \circ \theta \circ \chi \circ \iota. \tag{22}$$

Algorithm 3 describes Hsilu. The constant addition step ι adds a round constant C_i to x_0 of the state S in round i. The number of rounds is not fixed but depends on the concrete cryptographic use case. The round constants are numbered from the last round backwards (see Table 8). Consequently, the last round always utilizes the same round constant, regardless of the total number of rounds. The substitution layer χ is identical to that in Keccak. $\overline{x_i}$ denotes the bitwise complement of the word x_i.

Table 8: Round constants C_i, r is the number of rounds

i	Constant C_i	i	Constant C_i	i	Constant C_i
$r-12$	00000000000000f0	$r-8$	00000000000000b4	$r-4$	0000000000000078
$r-11$	00000000000000e1	$r-7$	00000000000000a5	$r-3$	0000000000000069
$r-10$	00000000000000d2	$r-6$	0000000000000096	$r-2$	000000000000005a
$r-9$	00000000000000c3	$r-5$	0000000000000087	$r-1$	000000000000004b

7.2 Search for Linear Layers Efficiently

There are still some undetermined parameters in our linear layer MRM, including the undetermined parameters in Table 7 and the parameters m_i and s_i. We determine these parameters using the method described in this section.

Algorithm 3 Definition of Hsilu

Parameters: Number q of rounds
for i from 0 to $q-1$ **do**
  $A = R_i(A)$
end for

The round function R_i:
$x_0 \leftarrow x_0 \oplus C_i$ ▷ ι
for j from 0 to 4 **do**
  $x_j = x_j \oplus (\overline{x_{j+1}} \odot x_{j+2})$ ▷ χ
end for
for j from 5 to $4+t$ **do**
  $x_j \leftarrow x_{n_1[j]} \oplus (x_{n_2[j]} \lll m_j)$ ▷ θ
end for
for j from t to $4+t$ **do**
  $x_j \leftarrow (x_j \lll s_j)$ ▷ ρ
end for

Our strategy is to randomly select parameters to generate a series of linear layers, and then exclude those that do not meet the required differential branch number. However, directly calculating the differential branch number of a linear layer is computationally slow, so we use filters before MILP. The overall process is outlined in Algorithm 4.

This algorithm consists of three main steps. Given the desired differential branch number b, N_{min} and N_{min}^{-1}, we first filter out linear layers that do not satisfy the requirements $N_{min}[i]$ and $N_{min}^{-1}[i]$ for $1 \leq i \leq 4$. Then we filter out the linear layers that do not satisfy the required bit differential branch number using MILP. Lastly, we choose the appropriate s_i so that the branch number of the linear layer meets the requirement. We don't look for the best candidate but stop at the first that is good enough.

In the first step, we iterate over input active bit counts of 1, 2, 3, and 4, and calculate the number of active output bits. Then we discard the bad linear layers. We do the same thing for the inverse of the linear layer. In the last step, we randomly choose s_i, or set s_i manually based on the active output bits' state to diffuse the active bits into different S-boxes. This step is feasible because the differential branch number is small compared to the 5×64 state. The active bits are sparse.

The complexity of the Gauss-Jordan elimination algorithm is $\mathbf{O}(n^3)$. The matrix M corresponding to the linear layer has a dimension of 320×320. Due to the cyclic property of the matrix, we just need to loop $5 \times \binom{319}{3}$ times when going through c_1, c_2, c_3, c_4 in the columns. In conclusion, the complexity of detecting a linear layer is $\mathbf{O}(n^3)$ when we do not consider the part of MILP. Most linear layers can be excluded without employing MILP, making this algorithm efficient.

The result of our search is shown in Table 9.

Table 9: The linear layer of Hsilu

i	5	6	7	8	9	10	11	12	13	14	15	16	17
$n_1[i]$	0	3	1	0	5	5	7	11	12	12	13	14	15
$n_2[i]$	4	4	2	1	2	3	6	8	9	6	10	11	9
m_i	47	51	54	30	4	55	0	14	59	4	10	45	53
s_i									0	57	13	31	20

Algorithm 4 Search for linear layers

Input: Desired branch number b, N_{min} and N_{min}^{-1}

for j from 0 to S_1 **do**
  Randomly select n_1, n_2, m_i and calculate the corresponding matrix M.
  $C = set\ of\ columns\ of\ M$
  for i from 1 to 4 **do**
    for $c_1, \cdots, c_i$ in C **do**
      if $weight(c_1 \oplus \cdots \oplus c_i) < N_{min}[i]$ **then**
        goto BadLinearLayer
      end if
    end for
  end for
  Calculate the inverse of M by Gauss-Jordan Elimination.
  Repeat the loop for $C' = set\ of\ columns\ of\ M^{-1}$.
  Solve bit branch number b' of the linear layer by MILP.
  if $b' < b$ **then**
    goto BadLinearLayer
  end if
  $b'' = 0$
  while $b'' < b$ **do**
    Select s_i and solve branch number b'' of linear layer by MILP.
  end while
  return (n_1, n_2, m_i, s_i)
  BadLinearLayer:
    continue
end for

7.3 MRM Variants on Other Numbers of XORs and Other Dimensions

In the previous analysis, we determined that the XOR count t is greater than or equal to $3r - 2$. When $r = 5$, we set $t = 13$. However, we can make adjustments to extend our design method to linear layers with different numbers of XORs. We consider linear layers with XOR counts of 12, 14, and 15.

For the linear layer with an XOR count of 12, if the XOR count for p_1 remains unchanged and the XOR count for p_2 is reduced by 1, this results in a reduction of the linear branch number to 4. Therefore, we can increase the XOR count of p_1 to 7, which will allow 4 positions of n_1 to exceed 4. Consequently, the XOR count for p_2 can be reduced to 5 without decreasing the linear branch number. This analysis is analogous to that in Section 6.2. This adjustment lowers the total number of XORs to 12, as illustrated in Table 10.

Table 10: The variant of MRM with 12 XORs

i	5	6	7	8	9	10	11	12	13	14	15	16
$n_1[i]$	{0,1,2,3,4}			{5,6,...,$i-1$}				7	12	13	14	15
$n_2[i]$	{0,1,2,3,4}							{8,9,10,11}				

For linear layers with XOR counts of 14 and 15, the adjustment method is straightforward: we maintain the XOR count for p_1 at 6 while increasing the XOR count for p_2, as shown in Table 11 and Table 12.

Table 11: The variant of MRM with 14 XORs

i	5	6	7	8	9	10	11	12	13	14	15	16	17	18
$n_1[i]$	{0,1,2,3,4}				5	5	{7,8,9,10}	11	12	13	14	15	16	17
$n_2[i]$	{0,1,2,3,4}						6	7	8	{6,7,8,9,10}	{9,10,11,12,13}			

Table 12: The variant of MRM with 15 XORs

i	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
$n_1[i]$	{0,1,2,3,4}				5	5	{7,8,9,10}	11	12	13	14	15	16	17	18
$n_2[i]$	{0,1,2,3,4}						6	7	8	{6,7,8,9,10}		{9,10,11,12,13,14}			

MRM can be easily applied to different dimensions. We utilized MILP to analyze the branch number of MRM with other XOR counts and other dimensions. The comparison of branch number is presented in Table 13. The instances of the linear layers are shown in Appendix A.

Table 13: Branch number of the variants of MRM. MRM-r means the number of rows of state is r.

Linear Layer	MRM-4		MRM-5					MRM-6
Dimension	4×64		5×64					6×64
XORs per bit	2.5	2.75	2.4	2.6	2.6	2.8	3	2.67
Differential branch number	9	12	10	10	12	13	14	13
Linear branch number	5	5	5	5	5	5	5	5

8 Results

In this section, we first compute the upper and lower bounds of the minimum weights of 3-round trails for Hsilu and compare Hsilu with other permutations to analyze their differential and linear security. Subsequently, we evaluate the hardware and software performance of Hsilu and compare with other permutations.

8.1 Trail Search and Bounds

In this section, we analyze the differential and linear trails of Hsilu and compare them with the trails of Ascon, Xoodoo, Gaston and Gaston-S.

8.1.1 Differential Trail

The method we utilize to calculate the lower bound of 3-round differential trail is similar to that in [DDLS24], which employs a lossy MILP encoding of the propagation rules based on the Hamming weight. Our method is more aggressive than that presented in [DDLS24]. We model the S-box using a lossy differential distribution table (DDT), as shown in Table 14. Although it is not a fully accurate DDT, the lower bound computed using this approximation is still meaningful. This approach provides a looser bound than the precise DDT, scaling the lower bound. Compared to the fully accurate representation of the S-box's DDT, we reduce the number of MILP constraints for the S-box from 26 to 4 and the number of constraints for the probabilities from 6 to 2. This strategy accelerates the MILP process, demonstrating that the lower bound for the differential trail weights in 3 rounds of Hsilu is at least 48.

Table 14: The lossy Hamming weight DDT of χ

hw	0	1	2	3	4	5
0	32	0	0	0	0	0
1	0	8	8	8	8	8
2	0	4	4	4	4	4
3	0	4	4	4	4	4
4	0	2	2	2	2	2
5	0	2	2	2	2	2

The upper bound of the minimum weight of 3-round trail is corresponding to the weight of the best-found 3-round trail. We utilize an MILP model to derive the activity pattern. The activity pattern of Hsilu is in Appendix C. Our analysis of the active patterns of Hsilu reveals that $N_{min}^{-1}[1]$ and $N_{min}^{-1}[2]$ are 54 and 36, while $N_{min}^{-1}[3]$ is only 13. Based on the analysis in Section 4, we hypothesize that when the number of active S-boxes in the second round is 3, a low-weight trail exists. Therefore, by fixing the number of active S-boxes in the second round at 3, we conducted a search using MILP and identified a trail with (13, 3, 25) active S-boxes and a weight of 84 (26, 6, 52). The trail is shown in Appendix B.1.

8.1.2 Linear Trail

We employed the method described in [MR22] to search for 3-round linear trails. Because the linear branch number is 5, we identified potential 3-round linear trails, as illustrated in Table 15. We categorized cases with a number of active S-boxes less than or equal to 18 into two groups: those with a number of active S-boxes less than or equal to 17 and those equal to 18. Through Satisfiability Modulo Theories (SMT), we proved that all potential trails are infeasible. Consequently, we determined that there are at least 19 active S-boxes, with a lower bound of minimum weight of 38. The lower bound of the minimum 3-round linear trail weight is greater than those of Ascon and Gaston.

Table 15: All possible configurations of active S-boxes for 3-round linear trail for $n \leq 18$. Here n denotes the sum of the number of active S-boxes. (d_0, d_1, d_2) denotes a trail with d_0, d_1 and d_2 active S-boxes in round 0, 1 and 2. $[a, b]$ denotes $a \leq d_2 \leq b$.

n	(d_0, d_1, d_2)
≤ 17	$(1, 4, \leq 12), (1, 5, \leq 11), \cdots, (1, 15, 1),$ $(2, 3, [2, 12]), (2, 4, \leq 11), \cdots, (2, 14, 1),$ $(3, 2, [3, 12]), (3, 3, [2, 11]), \cdots, (2, 14, 1),$ $\cdots,$ $(11, 1, [4, 5]), (11, 2, [3, 4]), \cdots, (11, 5, 1),$ $(12, 1, 4), (12, 2, 3), (12, 3, 2), (12, 4, 1)$
18	$(1, 4, 13), (1, 5, 12), \cdots, (1, 16, 1),$ $(2, 3, 13), (2, 4, 12), \cdots, (2, 15, 1),$ $(3, 2, 13), (3, 3, 12), \cdots, (3, 14, 1),$ $\ldots,$ $(13, 1, 4), (13, 2, 3), (13, 3, 2), (13, 4, 1)$

We employ the same method as in the Section 8.1.1 to calculate the upper bound by fixing the weight of the second round at a lower value. The best-found 3-round linear trail of Hsilu has $(16, 2, 6)$ active S-boxes and a weight of 52 $(36, 4, 12)$. The trail is shown in Appendix B.2.

We compare the branch number and minimum weight of trail of Hsilu with other permutations. The results are shown in Table 16.

Table 16: Branch number and minimum trail weight

	Ascon	Xoodoo	Gaston	Gaston-S	Hsilu
XORs per bit of linear layer	2†	2	3.2	4	2.6
Differential					
Branch number	4	4	12	12	10
2-round trail weight	8	8	24	24	20
Upper bound of 3-round trail weight	40	36	≤ 106	≤ 148	≤ 84
Lower bound of 3-round trail weight	40	36	-	-	≥ 48
Linear					
Branch number	4	4	4	12	5
2-round trail weight	8	8	8	24	10
Upper bound of 3-round trail weight	28	36	34	≤ 141	≤ 52
Lower bound of 3-round trail weight	28	36	34	-	≥ 38

† If we think of the χ as the non-linear layer of Ascon, the linear part of Ascon has a cost of 3.2 binary XOR operations per bit.

8.2 Implementation Efficiency of Hsilu

In this section, we evaluate the hardware and software performance of Hsilu and compare its performance with other permutations.

8.2.1 Hardware Performance

We implement Hsilu in Verilog. We use Openlane2 for synthesis. The synthesis strategy employed is AREA 0. We utilized an open-source PDK known as Skywater 130nm, which is a collaboration between Google and SkyWater Technology Foundry. The comparison with Ascon, Xoodoo, and Gaston is shown in Table 17.

Hsilu exhibits significant advantages over other permutations in terms of hardware area and gate count. In terms of latency, Hsilu outperforms both Ascon and Gaston.

Table 17: Hardware performance of Ascon, Xoodoo, Gaston, Hsilu.

	Dimension	Time delay (ns)	Area (μm^2)	Gates
Ascon	$5 \times 64, GF(2)$	4.6095	29014.0768	2098
Xoodoo	$3 \times 4 \times 32, GF(2)$	3.6495	29628.4160	1920
Gaston	$5 \times 64, GF(2)$	5.1800	35042.3584	2245
Hsilu	$5 \times 64, GF(2)$	4.2896	27526.4000	1792

8.2.2 Software Performance

We evaluated the cycles per round per byte for Hsilu on x86_64 and ARMv8 architectures. The comparison with Ascon, Xoodoo, Gaston and Gaston-S is shown in Table 18.

On the x86_64 architecture, Hsilu is the fastest of all these permutations. On the ARMv8 architecture, Hsilu is faster than Gaston, Gaston-S and Ascon but slower than Xoodoo.

Table 18: Cycles per round per byte of Ascon, Xoodoo, Gaston, Gaston-S, Hsilu.

	Ascon	Xoodoo	Gaston	Gaston-S	Hsilu
AMD EPYC 9754(x86_64)	0.3650	0.3768	0.5394	0.6010	0.3335
Neoverse-N1(ARMv8)	0.3141	0.2866	0.3956	0.4237	0.2981

9 Conclusions

In this work, we present a new linear layer MRM and permutation Hsilu aimed at low number of XORs while maintaining a high branch number. We study the properties of the inverse of linear layers with low number of XORs, as well as the differential and linear mask propagation properties. Based on these analyses, we determine the structure and parameters of the linear layer.

Our linear layer, MRM, achieves a low cost of 2.6 XORs per bit while having a differential branch number of 12 and a linear branch number of 5. The branch numbers outperform those of Gaston and Ascon. The 3-round linear trail of Hsilu has a minimum weight ranging from 38 to 52, surpassing Ascon's 28 and Gaston's 34. Hsilu outperforms Gaston and Ascon in terms of both software and hardware performance. Furthermore, we explore variants of MRM on different numbers of XORs and other dimensions, which also demonstrates good performance in terms of branch number.

Open Problems First, studying the relationship between the linear layer structure and the trail weight of more rounds is a challenging problem and the tighter bounds for the trail weight of more than two rounds remain unknown. Second, while we investigated how n_1 and n_2 influence the linear layer, we did not examine the impact of m. Third, it is an interesting research direction to apply MRM to other cryptographic structures and explore concrete use cases.

Acknowledgments

We would like to thank the anonymous reviewers for their constructive comments. This work was supported by the National Key R&D Program of China (No. 2024YFA1013000 and 2023YFB4503203), the National Natural Science Foundation of China (Grant No. 12231015 and 62122085), the Strategic Priority Research Program of the Chinese Academy of Sciences under Grant XDB0690000 and the Youth Innovation Promotion Association of Chinese Academy of Sciences.

References

[BBB+20] Davide Bellizia, Francesco Berti, Olivier Bronchain, Gaëtan Cassiers, Sébastien Duval, Chun Guo, Gregor Leander, Gaëtan Leurent, Itamar Levi, Charles Momin, Olivier Pereira, Thomas Peters, François-Xavier Standaert, Balazs Udvarhelyi, and Friedrich Wiemer. Spook: Sponge-based leakage-resistant authenticated encryption with a masked tweakable block cipher. *IACR Trans. Symm. Cryptol.*, 2020(S1):295–349, 2020.

[BDPA11] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. The Keccak reference, 2011.

[BJK+16] Christof Beierle, Jérémy Jean, Stefan Kölbl, Gregor Leander, Amir Moradi, Thomas Peyrin, Yu Sasaki, Pascal Sasdrich, and Siang Meng Sim. The SKINNY family of block ciphers and its low-latency variant MANTIS. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part II*, volume 9815 of *LNCS*, pages 123–153. Springer, Berlin, Heidelberg, August 2016.

[BKL+07] Andrey Bogdanov, Lars R. Knudsen, Gregor Leander, Christof Paar, Axel Poschmann, Matthew J. B. Robshaw, Yannick Seurin, and C. Vikkelsoe. PRESENT: An ultra-lightweight block cipher. In Pascal Paillier and Ingrid

Verbauwhede, editors, *CHES 2007*, volume 4727 of *LNCS*, pages 450–466. Springer, Berlin, Heidelberg, September 2007.

[BPP+17] Subhadeep Banik, Sumit Kumar Pandey, Thomas Peyrin, Yu Sasaki, Siang Meng Sim, and Yosuke Todo. GIFT: A small present - towards reaching the limit of lightweight encryption. In Wieland Fischer and Naofumi Homma, editors, *CHES 2017*, volume 10529 of *LNCS*, pages 321–345. Springer, Cham, September 2017.

[BS91] Eli Biham and Adi Shamir. Differential cryptanalysis of DES-like cryptosystems. *Journal of Cryptology*, 4(1):3–72, January 1991.

[DDLS24] Mathieu Degré, Patrick Derbez, Lucie Lahaye, and André Schrottenloher. New models for the cryptanalysis of ASCON. Cryptology ePrint Archive, Report 2024/298, 2024.

[DEMS21] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schläffer. Ascon v1.2: Lightweight authenticated encryption and hashing. *Journal of Cryptology*, 34(3):33, July 2021.

[DHAK18] Joan Daemen, Seth Hoffert, Gilles Van Assche, and Ronny Van Keer. The design of Xoodoo and Xoofff. *IACR Trans. Symm. Cryptol.*, 2018(4):1–38, 2018.

[DL18] Sébastien Duval and Gaëtan Leurent. MDS matrices with lightweight circuits. *IACR Trans. Symm. Cryptol.*, 2018(2):48–78, 2018.

[DR01] Joan Daemen and Vincent Rijmen. The wide trail design strategy. In Bahram Honary, editor, *8th IMA International Conference on Cryptography and Coding*, volume 2260 of *LNCS*, pages 222–238. Springer, Berlin, Heidelberg, December 2001.

[DR02] Joan Daemen and Vincent Rijmen. *The Design of Rijndael: AES - The Advanced Encryption Standard*. Information Security and Cryptography. Springer, 2002.

[HDRM23] Solane El Hirch, Joan Daemen, Raghvendra Rohit, and Rusydi H. Makarim. Twin column parity mixers and gaston - A new mixing layer and permutation. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part III*, volume 14083 of *LNCS*, pages 475–506. Springer, Cham, August 2023.

[KLSW17] Thorsten Kranz, Gregor Leander, Ko Stoffelen, and Friedrich Wiemer. Shorter linear straight-line programs for MDS matrices. *IACR Trans. Symm. Cryptol.*, 2017(4):188–211, 2017.

[LP24] Gaëtan Leurent and Clara Pernot. Design of a linear layer optimised for bitsliced 32-bit implementation. *IACR Trans. Symm. Cryptol.*, 2024(1):441–458, 2024.

[LRL+24] Hao Lei, Raghvendra Rohit, Guoxiao Liu, Jiahui He, Mohamed Rachidi, Keting Jia, Kai Hu, and Meiqin Wang. Symmetric twin column parity mixers and their applications. *IACR Transactions on Symmetric Cryptology*, 2024(4):1–37, Dec. 2024.

[Mat94] Mitsuru Matsui. Linear cryptanalysis method for DES cipher. In Tor Helleseth, editor, *EUROCRYPT'93*, volume 765 of *LNCS*, pages 386–397. Springer, Berlin, Heidelberg, May 1994.

[MR22] Rusydi H. Makarim and Raghvendra Rohit. Towards tight differential bounds of Ascon A hybrid usage of SMT and MILP. *IACR Trans. Symm. Cryptol.*, 2022(3):303–340, 2022.

[SD18] Ko Stoffelen and Joan Daemen. Column parity mixers. *IACR Trans. Symm. Cryptol.*, 2018(1):126–159, 2018.

[ZBRL15] Wentao Zhang, Zhenzhen Bao, Vincent Rijmen, and Meicheng Liu. A new classification of 4-bit optimal S-boxes and its application to PRESENT, RECTANGLE and SPONGENT. In Gregor Leander, editor, *FSE 2015*, volume 9054 of *LNCS*, pages 494–515. Springer, Berlin, Heidelberg, March 2015.

A The Variant of MRM

Table 19: The variant of MRM-5 with 12 XOR operations

i	5	6	7	8	9	10	11	12	13	14	15	16
$n_1[i]$	1	3	2	5	5	6	9	10	12	13	14	15
$n_2[i]$	4	4	0	1	0	2	3	7	11	8	10	9
m_i	35	17	1	15	55	15	44	3	33	61	22	21
s_i								0	28	26	45	13

Table 20: The variant of MRM-5 with 14 XOR operations

i	5	6	7	8	9	10	11	12	13	14	15	16	17	18
$n_1[i]$	1	2	2	0	5	5	6	11	12	13	14	15	16	17
$n_2[i]$	4	3	1	3	0	4	9	7	8	6	10	12	13	9
m_i	12	22	43	34	38	14	49	33	46	14	48	26	16	39
s_i										0	58	15	38	22

Table 21: The variant of MRM-5 with 15 XOR operations

i	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
$n_1[i]$	1	2	2	0	5	5	6	11	12	13	14	15	16	17	18
$n_2[i]$	4	3	1	3	0	4	9	7	8	6	7	11	10	14	9
m_i	56	2	16	34	28	55	7	25	17	9	5	22	20	17	30
s_i											0	40	60	21	5

Table 22: The variant of MRM-4 with 10 XOR operations

i	4	5	6	7	8	9	10	11	12	13
$n_1[i]$	2	2	3	4	4	5	9	10	11	12
$n_2[i]$	1	0	0	1	3	8	6	9	7	8
m_i	19	35	40	40	27	7	60	58	21	26
s_i							0	13	50	11

Table 23: The variant of MRM-4 with 11 XOR operations

i	4	5	6	7	8	9	10	11	12	13	14
$n_1[i]$	0	3	0	4	4	5	9	10	11	12	13
$n_2[i]$	3	2	1	1	2	7	8	6	7	10	9
m_i	38	41	50	36	22	23	13	28	8	54	4
s_i								0	33	24	35

Table 24: The variant of MRM-6 with 16 XOR operations

i	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
$n_1[i]$	2	2	0	5	0	6	6	7	13	14	15	16	17	18	19	20
$n_2[i]$	3	1	4	3	5	4	1	11	8	9	10	12	15	14	11	13
m_i	43	59	21	38	15	5	38	62	59	60	62	60	4	16	42	61
s_i											0	15	48	63	6	33

B Differential and Linear Trails of Hsilu

B.1 Differential Trail of Hsilu

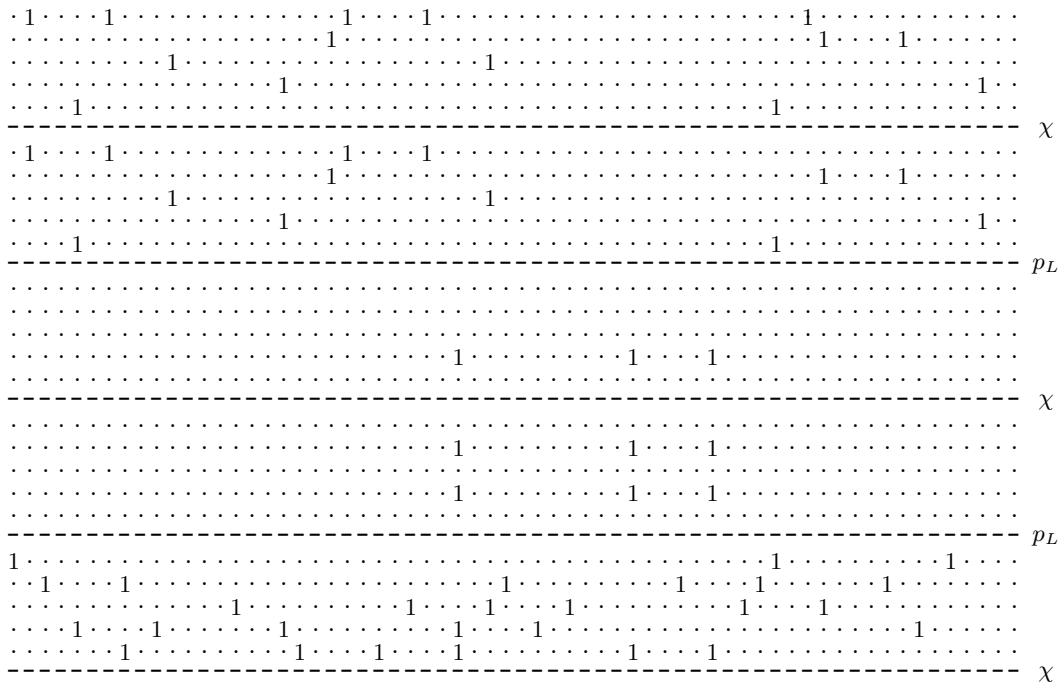

Figure 1: 3-round differential trail with (13, 3, 25) active S-boxes and weight 84 (26, 6, 52). We omit the third round in the figure.

B.2 Linear Trail of Hsilu

Figure 2: 3-round linear trail with (16, 2, 6) active S-boxes and weight 52 (36, 4, 12). We omit the third round in the figure.

C Activity pattern of MRM in Hsilu

Table 25: Activity pattern of MRM in Hsilu

d	$N_{min}[d]$	$N_{max}[d]$	$N_{min}^{-1}[d]$	$N_{max}^{-1}[d]$	d	$N_{min}[d]$	$N_{max}[d]$	$N_{min}^{-1}[d]$	$N_{max}^{-1}[d]$
1	9	37	54	64	33	3	64	2	64
2	8	57	36	64	34	3	64	2	64
3	7	64	13	64	35	2	64	2	64
4	6	64	8	64	36	3	64	1	64
5	6	64	7	64	37	3	64	1	64
6	6	64	4	64	38	3	64	2	64
7	5	64	3	64	39	3	64	2	64
8	4	64	2	64	40	2	64	2	64
9	6	64	1	64	41	3	64	2	64
10	5	64	1	64	42	3	64	2	64
11	6	64	1	64	43	2	64	2	64
12	6	64	2	64	44	2	64	2	64
13	3	64	2	64	45	2	64	2	64
14	6	64	1	64	46	2	64	2	64
15	6	64	2	64	47	2	64	2	64
16	4	64	2	64	48	2	64	2	64
17	5	64	1	64	49	2	64	2	64
18	5	64	1	64	50	2	64	2	64
19	3	64	1	64	51	2	64	2	64
20	4	64	1	64	52	2	64	2	64
21	4	64	1	64	53	2	64	2	64
22	3	64	1	64	54	2	64	2	64
23	4	64	2	64	55	2	64	2	64
24	3	64	1	64	56	2	64	2	64
25	3	64	2	64	57	2	64	2	64
26	3	64	2	64	58	1	64	3	64
27	3	64	1	64	59	2	64	3	64
28	3	64	1	64	60	1	64	3	64
29	4	64	2	64	61	1	64	3	64
30	4	64	1	64	62	1	64	3	64
31	3	64	1	64	63	1	64	3	64
32	3	64	2	64	64	1	64	3	64